

Deep Reinforcement Learning Project Report

Dhruv Parsewar

1 Description of the Problem Statement

As discussed earlier in classical reinforcement learning, agents use a simple spreadsheet (called a Q-table) to memorize the best move for every single square on a grid. This works great for simple games. However not for real-world problems.

We needed to replace this spreadsheet with a “brain” a Neural Network. The problem is that plugging a neural network directly into a live learning agent usually makes it crash. The agent gets confused it likely forgets older lessons and it constantly updates its own goals

The goal of this project was to build a stable neural network agent from scratch and prove it works by having it solve a game called **FrozenLake**, where the agent must navigate across a grid of ice without falling into holes.

2 Proposed Solution

To solve the crashing and instability issues, I built a Deep Q-Network (DQN) from the ground up using three specific features:

- **The Brain (Neural Network):** The agent uses a neural network that looks at its current position on the ice and predicts a score for moving Left, Down, Right, or Up. It picks the direction with the highest score.
- **The Memory (Experience Replay):** Instead of learning from one step at a time (which causes it to forget past mistakes), the agent saves every single move it makes into a giant memory. During training, it pulls out a random handful of past memories to study. This mixes up the data and keeps the math stable.
- **The Anchor (Target Network):** To solve the “moving target” problem, I created a second, frozen copy of the neural network. The agent uses this frozen copy strictly to calculate the score it should be aiming for.
- **Reward Shaping:** The default game only gives a reward if you perfectly reach the end. To help the blind agent learn faster, I gave it a small penalty for every normal step it took and a large penalty for falling into a hole. This created a trail of clues for the neural network to follow.

3 Results

The custom agent was tested in two different versions of the FrozenLake game to prove it was robust.

- **1) Regular Ice (Deterministic)**

- I trained the agent for 600 rounds. For the first 100 rounds, the agent just stumbled around randomly, constantly falling into holes (a 0% win rate).
- Once it randomly stumbled upon the goal, its memory bank helped it learn the perfect path backwards.
- By round 400, it achieved a **80-90% win rate**. The reason it didn't hit 100% is because I hardcoded a rule forcing the agent to make a random exploratory move 5% of the time to ensure it never gets stuck.

- **2) Slippery Ice (Stochastic)**

- Next, I turned on the “slippery” physics, meaning when the agent tries to go right, it will randomly slide up or down 66% of the time.
- Without changing a single line of the agent's code, I let it train for 3,000 rounds. The agent successfully adapted to the random sliding. It learned a highly cautious strategy, hugging the walls to avoid sliding into holes, proving the underlying algorithm works even in unpredictable environments.

4 Code Overview

The code was written in Python using PyTorch to handle the neural network math. The most critical part of the code happens inside the `train_step` function, which acts as the agent's learning cycle.

Here is what the code does step-by-step during training:

1. **Make a Guess:** The agent grabs a batch of random memories and asks its primary brain to guess the value of the moves it made.
2. **Check the Anchor:** It asks the frozen Target Network what the *actual* target score should be for those moves.
3. **Handle Game Overs:** If the memory shows the agent falling in a hole or winning the game, the code uses a mathematical mask (multiplying by zero) to ensure no future scores are added to that specific path.
4. **Update the Brain:** The code calculates the difference (the error) between the agent's guess and the Anchor's target. It then uses gradient descent to adjust the weights of the primary brain, making it slightly smarter for the next round.